Алгебра логарифма отношения правдоподобия

Оглавление

Алгебра логарифма отношения правдоподобия....1
Введение....1
Природа LLR....4
Пример расчёта LLR для стандарта КАМ-16....7
Пример использования корректирующего кода....10
Рекурсивное мягкое декодирование кодов Рида-Маллера....10
Декодирование полярных кодов....16
Алгоритм Successive Cancellation, SC....16
Ссылка на код симулятора AWGN+QAM-16+Polar/Reed-Muller....24
Приложение А Расчёт полярной последовательности....25
Приложение Б Математика циклического префикса. MMSE эквалайзер и согласованный фильтр....30
Данный материал рекомендуется для уровня магистратуры технических вузов 👨‍🎓.

Введение

В цифровых системах передачи информации в процессе приёма активно используется такая метрика как *отношение правдоподобия*, чаще его натуральный логарифм, который обозначается как LLR (*Log Likelihood Ratio*). Данная необходимость вызвана стремлением извлечь (насколько это возможно) наибольшее количество информации в процессе приёма и, следовательно, получить минимальную вероятность ошибки, а значит и наибольшую достоверность приёма.

Величина LLR, L, как можно догадаться, является вещественным числом, $L \in \mathbb{R}$, что говорит о так называемой «мягкости» приёма (*soft-decision reception*). Мягкость подразумевает наличие нескольких уровней различения — более двух. В случае «жёсткого» приёма (*hard-decision*) уровней два, что соответствует биту — базовой единице информации. Для упрощения рассуждений будем подразумевать приём именно бита — двоичного символа, чтобы не зависеть от конкретного вида модуляции; это же справедливо и для корректирующего кода. В итоге, любой сложный приём можно свести к приёму бита, равно как и *количество информации* можно выразить в битах на двоичный символ. Заметим, что количество информации теоретически определяется как логарифм обратной вероятности, что намекает на единую природу с LLR.

Цифровой приём можно условно разделить на следующие этапы[1]:

- Аналого-цифровое преобразование (АЦП) с переносом на промежуточную или нулевую частоту. Результат: комплексный цифровой сигнал с частотой дискретизации выше символьной скорости (обычно с оверсэмплингом 2 или 4). Также здесь осуществляется грубая автоматическая регулировка усиления, АРУ, реализованная по типу *Closed-Loop* систем, то есть с петлёй обратной связи. АРУ защищает АЦП от клиппинга (перегрузки).

1 Ограничимся только SC-FDE (*Single-Carrier Frequency-Domain Equalization*) системами — системами на одной поднесущей с выравниванием канала (эквализацией) в частотной области.

- Первичная синхронизация (кадровая и символьная). Поиск преамбулы или известных пилот-последовательностей (например, через взаимно-корреляционную функцию, *Open-Loop* системы — без петли обратной связи). Определяются точные границы кадров и оптимальные моменты стробирования символов. Результат: избавление от неопределённости по времени во входном потоке отсчётов. Заметим, что эти данные далее используются для точной символьной синхронизации, которая строится с петлёй обратной связи (*Closed-Loop* системы). Этот принцип — сначала грубо, потом точно, используется довольно-таки часто в разных подсистемах систем связи и является общим принципом построения сложных автоматических систем.
- Цифровая интерполяция (интерполятор Фэрроу, *Farrow*), которая на этапе первичной синхронизации работает в фиксированном режиме, а после установления которой интерполятор переходит в режим слежения для компенсации возможного расхождения генераторов, *Clock Drift*.
- Грубая коррекция частотного сдвига. Оценка сильного смещения несущей частоты, вызванного эффектом Доплера или расхождением кварцевых генераторов. Коррекция выполняется путём умножения сигнала на соответствующую комплексную экспоненту, которая оценивается благодаря специально сформированной преамбуле, состоящей, например, из двух одинаковых сигналов [*A*, *A*] (алгоритм Шмидла-Кокса, *Open-Loop* системы).
- Удаление циклического префикса (*CP Removal*) для устранения межблочной интерференции (*Inter-Block Interference*, IBI), а также превращения линейной свёртки *импульсной характеристики* (ИХ) канала с переданным сигналом в циклическую (круговую), чтобы последующая коррекция канала в частотной области работала по точечно (т. е. для каждой гармоники независимо), а значит максимально просто.
- Частотно-областное выравнивание (эквалайзирование), *Frequency-Domain Equalization*, FDE. Устраняются искажения многолучёвости канала связи. Сигнал разбивается на блоки и затем результат БПФ (FFT) корректируется[2] частотной оценкой канала связи, полученной по пилот-последовательностям, например, *последовательностям Задова-Чу* или М-последовательностям. После коррекции сигнал переводится обратно во временную область. Результат: выравнивается амплитуда и фаза сигнала. Заметим, что сюда **входит согласованная с формирующим фильтром (например, Root-Raised Cosine) фильтрация**, которая даёт наибольшее ОСШ и нулевую *межсимвольную интерференцию*, МСИ. Для упрощения описания всю подсистему FDE можно называть согласованным фильтром, т. к. последний является ядром оптимального приёма (он «вытягивает» энергию) и должен использоваться всегда вне зависимости от наличия канальных искажений. При наличии канальных искажений коррекцию и согласованную фильтрацию выгодно делать в частотной области одновременно, правда для этого приходится вставлять циклический префикс (платить избыточностью), который и позволяет делать такую сложную фильтрацию[3] оптимально, используя достаточно простой алгоритм

2 Алгоритм минимального среднеквадратичного отклонения, MMSE.

3 Фактически деконволюцию импульсной характеристики канала совместно с классической согласованной фильтрацией.

(Приложение Б Математика циклического префикса. MMSE эквалайзер и согласованный фильтр).

- Энергетическое выравнивание — приведение принятого созвездия к эталонной сетке, используя, например, скользящее среднее значение текущей энергии. Результат: приведение энергии принимаемого сигнала к цифровой единице (или к любой наперёд заданной константе). Данную *Open-Loop* подсистему, которая работает при первоначальной синхронизации, дополняют *Closed-Loop* подсистемой для более точной подстройки уровня после установления синхронизации.
- Петля обратной связи (*Closed-Loop*) для цифрового интерполятора, в которую входят детектор ошибки времени (*Time Error Detector*, TED) и петлевой фильтр. Предназначена для максимального раскрытия глазковой диаграммы и поддержания этого раскрытия.
- Точная фазовая синхронизация (*Closed-Loop*). Устраняются остаточные фазовые шумы и медленный дрейф частоты, которые эквалайзер не смог убрать. Используются алгоритмы слежения (например, петля Костаса). Результат: принятое созвездие «стоит намертво».
- Формирование LLR-потока (*геометрический демаппинг* плюс взвешивание канала) — преобразование выровненных комплексных чисел в набор вещественных LLR согласно геометрии созвездия и уровня шума в канале.
- Декодирование — уточнение LLR согласно логике корректирующего кода и принятие решения о битах.

В данном пособии важен этап формирования LLR-потока. Здесь символьный поток (поток комплексных чисел) преобразуется в поток LLR — вещественных чисел, частота следования которых совпадает с частотой следования канальных (т. е. кодированных) битов. Для этого первым делом осуществляется геометрический демаппинг, который заметно упрощается благодаря использованию *двумерного кода Грея* для модуляций КАМ-М, а вторым делом осуществляется так называемое взвешивание канала. Последнее — это, по сути, амплитудное масштабирование отсчётов с выхода демаппера величиной $1/\sigma^2$, где σ^2 — уровень шума в канале или, попросту говоря, дисперсия шума в одной квадратуре[4]. Взвешивание канала необходимо для выравнивания LLR, особенно если используются длинные корректирующие коды — а именно такие используют в down-link канале передачи пользовательских данных 4G/5G. В этом случае разные кодовые биты могут прийти из разных каналов по разным причинам: разные антенны, разные лучи, разные отрезки времени, разные частотные подканалы и т. п.

На последнем (кодовом) этапе цифрового приёма происходит декодирование выровненных LLR в соответствии с выбранным корректирующим кодом. Декодирование заключается в очистке принятых LLR и последующем принятии решения о переданных битах. Если помехоустойчивое кодирование отсутствует, то результат демодуляции фактически превращается в битовый поток путём простого сравнения LLR с пороговым уровнем (как правило, нулём).

4 В нормально работающем и настроенном приёмнике уровень шума должен быть одинаковым для обеих квадратур.

Природа LLR

Определим что такое LLR, и почему он прямым образом вычисляется в процессе демаппинга — цифровой демодуляции.

Отношением правдоподобия γ называют отношение вероятностей в пользу двух априорно известных событий, правдоподобность которых мы хотим сравнить, проведя некий *опыт* (как правило это некое **измерение**). Итогом опыта (*a posteriori*) должно быть принятие решения относительно одного из двух событий. Поскольку в цифровой связи рассматривается приём битов (не зря мы говорили ранее, что любой сложный приём можно свести к приёму битов), то **достаточно** рассмотреть отношение вероятностей типа:

$$\gamma = \frac{p(0/\text{опыт})}{p(1/\text{опыт})}.$$

Здесь в числителе стоит вероятность в пользу бита 0, а в знаменателе — вероятность в пользу бита 1. Кто из них будет в числителе, а кто в знаменателе — дело выбора, но раз определившись, далее необходимо следовать этому выбору; стандартами связи принято бит ноль ставить в числитель, а также кодировать ноль положительным вещественным числом +1 (в процедуре мягкого декодирования кодов).

Отношению правдоподобия $\gamma = 1$ соответствует максимальная неопределённость, когда две вероятности равны между собой, и по необходимости при этом равны по ½ из-за *условия нормировки*.

Так как мы уже знаем, что всякому принятому биту предшествует некое вещественное число[5], то отношение правдоподобия на самом деле записано именно для числа v, которое является условием, а фигурирующие вероятности — условными (послеопытными):

$$\gamma(v) = \frac{p(0/v)}{p(1/v)}.$$

Вещественное число v — это результат измерения (опыта), по которому мы должны в итоге принять решение: 0 это или 1. Если вычисленное $\gamma > 1$, то принимается решение о бите 0; в противном случае решается, что это бит 1. Такое решение называют решением по критерию *максимума апостериорной вероятности* (*Maximum A Posteriori Decision, MAP decision*).

Так как отношение вероятностей имеет очень большую вариативность, рассматривают натуральный *логарифм отношения правдоподобия* (LLR):

$$L(v) = \ln \gamma(v).$$

Величина LLR по модулю определяет степень уверенности в событии: чем выше LLR, тем больше уверенности. Максимальную уверенность обозначают как inf (бесконечность). При практическом использовании inf вполне определена форматом используемых чисел как форматный максимум (при fixed-point арифметике максимум может быть равен +1.0). Знак LLR определяет бит: если знак плюс, то LLR декодируется (интерпретируется) как бит 0; если знак минус, то LLR декодируется как бит 1.

---

5 Re- или Im-часть с выхода согласованного фильтра — так мы называем фильтр, который убирает разного рода интерференцию и даёт наибольшее ОСШ.

Учтя условие нормировки вероятностей, послеопытные вероятности можно выразить через LLR:

$$p(0/v)=\frac{e^{L(v)}}{1+e^{L(v)}},\quad p(1/v)=\frac{1}{1+e^{L(v)}},$$

что пригодится далее при построении алгебры LLR.

По физическим причинам всякое измеренное вещественное число v состоит из сигнальной компоненты, s, и шумовой, n:

$$v=s+n.$$

Сигнальная компонента на выходе согласованного фильтра не является случайной и определяется переданным символом. Предполагается, что канальные искажения полностью скомпенсированы эквалайзером, а тактовая синхронизация успешно установлена. В связи с этим все остаточные неустранимые эффекты можно трактовать в виде аддитивной шумовой компоненты. Поскольку при штатной работе приёмника данная компонента формируется в результате суперпозиции множества независимых факторов с соизмеримыми дисперсиями, формируемый шум, согласно центральной предельной теореме, с высокой точностью аппроксимируется гауссовым распределением. Спектральные характеристики этого шума в пределах рабочей полосы частот приёмника позволяют рассматривать его как белый шум. Именно поэтому классической базовой моделью для анализа подобных систем является канал с аддитивным белым гауссовским шумом (АБГШ).

Заметим, что согласованная фильтрация обеспечивает наибольшее выходное отношение сигнал-шум, что эквивалентно минимальной дисперсии шума при фиксированном (нормированном) уровне сигнала. Таким образом, все звенья тракта до согласованного фильтра направлены на устранение специфических канальных и аппаратурных искажений, а также на минимизацию погрешностей преобразования и синхронизации, в то время как сам согласованный фильтр минимизирует неустранимый остаточный шум[6].

Не умаляя общности, запишем *плотность распределения вероятностей* (ПРВ) величины v как $w(v)$, подразумевая под шумом n шум с нулевым средним и дисперсией σ^2. Тогда среднее значение рассматриваемой величины будет в точности равно сигнальной компоненте:

$$\mu_v=\overline{v}=s,$$

а её дисперсия — дисперсии шумовой компоненты:

$$D_v=\overline{(v-\mu_v)^2}=\overline{n^2}=\sigma^2.$$

Введённое ранее отношение правдоподобия можно выразить через отношение условных ПРВ:

$$\gamma(v)=\frac{p(0)}{p(1)}\frac{w(v/0)}{w(v/1)}.$$

Здесь вероятности $p(0)$ и $p(1)$ — безусловные (априорные) вероятности битов. В системах цифровой связи эти вероятности равны друг другу (хотя бы за счёт

6 Белый гауссов шум имеет максимальную энтропию по Шеннону.

скремблирования данных). Учитывая гауссов закон ПРВ, а также беря натуральный логарифм данного отношения, получим:

$$L(v)=\frac{(v-v_1)^2-(v-v_0)^2}{2\sigma^2}=\frac{(v_0-v_1)}{\sigma^2}\left(v-\frac{v_0+v_1}{2}\right).$$

Здесь v_0 — среднее значение v при условии, что передавался бит 0, v_1 — среднее значение при условии, что передавался бит 1. Их полусумме соответствует оптимальный пороговый уровень (вспомним про $\gamma=1$), которому соответствует **минимальная вероятность ошибки приёма**. В общем случае для минимизации вероятности ошибки приёма в качестве порогового уровня следует выбрать точку пересечения двух условных ПРВ.

Рассмотрим некоторые крайние случаи.

- Если измеренное значение v равно v_0, то:

$$L(v_0)=\frac{(v_0-v_1)^2}{2\sigma^2}>0.$$

- Если же измеренное значение v равно v_1, то:

$$L(v_1)=\frac{-(v_0-v_1)^2}{2\sigma^2}=-L(v_0)<0.$$

Квадрат разности, $(v_0-v_1)^2$, называют квадратом расстояния между так называемыми *виртуальными* битами. Также его называют *энергией разностного сигнала*. Приём (т. е. различение) принципиально возможен, если $v_0\neq v_1$, а это возможно только если точки созвездия отличаются друг от друга.

Отношение:

$$q^2\stackrel{\text{def}}{=}\frac{(v_0-v_1)^2}{\sigma^2}$$

называют *отношением сигнал-шум после обработки*. Его зачастую записывают в каноническом виде:

$$q^2=\frac{2E_d}{N_0}.$$

Здесь N_0 — *спектральная плотность мощности* (СПМ) «аналогового» шума, E_d — энергия разностного сигнала. Если бы мы рассматривали *фазовую манипуляцию* с двумя состояниями (ФМн-2), то разностным сигналом была бы разность сигналов для битов 0 и 1. Для более сложных модуляций (m-битовых) как такового разностного сигнала нет, и можно лишь сопоставить эквивалентный разностный сигнал (виртуальный), чтобы была возможность сравнивать с ФМн-2, которая является базовой цифровой модуляцией. Под аналоговым шумом со СПМ N_0 понимается белый шум $n(t)$ на входе согласованного фильтра[7].

7 На практике шум может быть и цифровым, однако, это не меняет сути теории оптимального приёма, и даёт лишь некоторую погрешность, то есть является вторичным.

Таким образом, выяснили, что LLR по модулю прямо пропорционален измеренному вещественному значению v, делённому на уровень шума σ^2; по знаку же — при склонности к биту 0 знак положительный, а при склонности к биту 1 — отрицательный.

Пример расчёта LLR для стандарта КАМ-16

Рассмотрим созвездие модуляции КАМ-16 из стандарта 3GPP на беспроводную связь Wi-Fi.

Для четырёх битов одного символа модуляции можно записать следующие выражения LLR:

$$L_{b_0}=4\,I/\sigma^2,\quad L_{b_1}=4\,Q/\sigma^2,\quad L_{b_2}=2\left(2-|I|\right)/\sigma^2,\quad L_{b_3}=2\left(2-|Q|\right)/\sigma^2.$$

Поскольку LLR первых двух битов (при прочих равных условиях; например, при $I=Q=1$) в два раза больше LLR последних двух, то первые два бита защищены от шума лучше. Так как формулы для расчёта битовых LLR независимые, то демодуляция при выбранной кодировке выполняется параллельно. Поясним вывод полученных формул для битовых LLR.

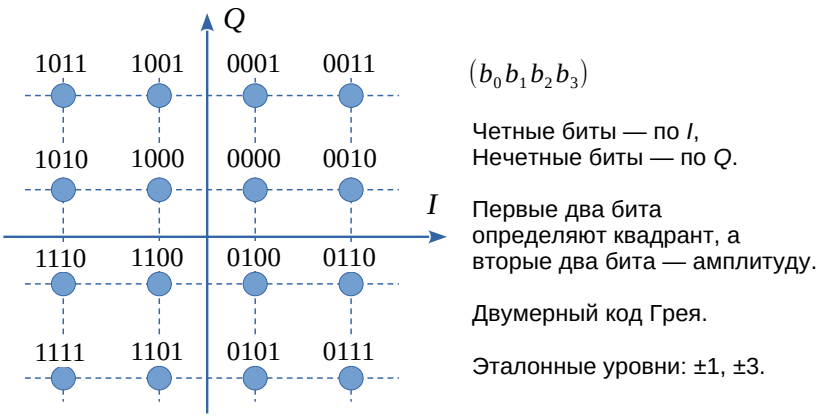


Рисунок 1: Созвездие КАМ-16 из стандарта 3GPP Wi-Fi

Так как первый бит определяется квадратурой I, то вещественному числу v в приёмнике будет соответствовать ось I с пороговым уровнем 0. Величинами v_0 и v_1 будут при этом $+2$ и -2, соответственно; это некие *виртуальные точки*, которые были определены из следующих соображений. Например, для бита 1 эталонными уровнями будут -1 и -3, но так как эти уровни равновероятные, то среднее значение будет равно -2 — именно оно использовалось при расчёте LLR по основной формуле (смотри выше).

Для третьего бита осью вещественного числа v будет ось I с пороговыми уровнями либо -2, либо 2 в зависимости от номера квадранта, II и I, соответственно. Величинами v_0 и v_1 будут при этом -3 и -1, либо 3 и 1, соответственно (здесь вместо усреднения применён модуль, что дало единую виртуальную точку, определяемую вне зависимости от остальных битов).

Вычислим вероятности ошибок для разных битов, используя общепринятую функцию ошибок erf(x).

Для первого и второго битов:

$$p_{b_0}=p_{b_1}=\frac{1}{2}\int_0^\infty w\left(v,-1,\sigma\right)dv+\frac{1}{2}\int_0^\infty w\left(v,-3,\sigma\right)dv=\frac{1}{2}-\frac{1}{4}\operatorname{erf}\left(3/\sqrt{2}/\sigma\right)-\frac{1}{4}\operatorname{erf}\left(1/\sqrt{2}/\sigma\right).$$

Для третьего и четвертого битов:

$$p_{b_2}=p_{b_3}=1/2\int_{-2}^{2}w(v,-3,\sigma)dv+1/2\int_{-\infty}^{-2}w(v,-1,\sigma)dv+1/2\int_{2}^{\infty}w(v,-1,\sigma)dv=$$
$$=\frac{1}{2}-\frac{1}{2}\operatorname{erf}(1/\sigma/\sqrt{2})-\frac{1}{4}\operatorname{erf}(3/\sigma/\sqrt{2})+\frac{1}{4}\operatorname{erf}(5/\sigma/\sqrt{2})$$

Если мы посчитаем среднее значение энергии **разностного** сигнала, то получим среднее ОСШ для рассматриваемой КАМ-16 (вспомним про виртуальные точки, усредним квадрат разности):

$$q^2=\frac{1}{2}\frac{(2-(-2))^2+(3-1)^2}{\sigma^2}=\frac{10}{\sigma^2}.$$

Заметим, что величине 10 соответствует известная в стандарте 3GPP нормировка амплитуд на корень из десяти (для КАМ-16), которая необходима для приведения энергии сигнала к единичному уровню. Это позволяет, в частности, формировать на передатчике сигнал фиксированной средней мощности **вне зависимости** от выбранной модуляции.

Учитывая, что все биты символа модуляции равновероятные, запишем итоговую битовую вероятность ошибки демодуляции КАМ-16 (3GPP Wi-Fi):

$$p_b=\frac{1}{2}-\frac{3}{8}\operatorname{erf}(q/2/\sqrt{5})-\frac{1}{4}\operatorname{erf}(3q/2/\sqrt{5})+\frac{1}{8}\operatorname{erf}(5q/2/\sqrt{5}).$$

Запишем для КАМ-16 каноническое отношение энергии на один бит к СПМ белого шума через среднее ОСШ q^2 (эти отношения сигнал-шум не стоит путать):

$$\frac{E_b}{N_0}=\frac{(2\cdot4+10\cdot8+18\cdot4)}{16\cdot\log_2 16}\frac{T}{2T\sigma^2}=\frac{5}{4\sigma^2}=\frac{q^2}{8}.$$

Здесь использовалось соотношение между СПМ входного шума и дисперсией шума на выходе согласованного фильтра типа «корень из приподнятого косинуса» при выборе нормировки *по постоянному току*[8] (то есть при $|H(0)|=1$, *Unity Gain*):

$$\sigma^2=\frac{N_0}{2T},$$

где T — интервал следования символов модуляции. Что касается величины E_b, то она была посчитана на выходе согласованного фильтра через средний квадрат модуля амплитуд точек созвездия КАМ-16 и деления на 4 (потому что для КАМ-16 имеем 4 бита на один символ) с последующим умножением на интервал следования символов (энергия равна средней мощности символа, умноженной на длительность символа).

В общем же случае следует учесть порядок модуляции КАМ-M и наличие корректирующего кода:

$$\frac{E_b}{N_0}=\frac{E_s}{2RT\sigma^2\log_2 M},$$

8 Есть ещё нормировка на единичную энергию (*Unity Energy*). Достоинство: не надо пересчитывать дисперсию шума после этапа фильтрации.

где E_s — средняя энергия созвездия КАМ-М, R — скорость кодирования. Здесь E_s также называется энергией на один символ модуляции. Часто энергию берут **в расчёте на единичный символьный интервал**, тогда величину T учитывать не требуется (на ОСШ она всё равно не влияет), поэтому в итоге можно записать более простые и общепринятые соотношения:

$$SNR_{\text{linear}} \stackrel{\text{def}}{=} \frac{E_s}{N_0} = \frac{E_s}{2\sigma^2}, \quad E_s = E_b R \log_2 M, \quad N_0 = 2\sigma^2.$$

При введении кодирования мы в один временной интервал помещаем больше кодовых битов, среди которых лишь часть полезных (пользовательских), поэтому скорость передачи информации падает, а на передачу одного пользовательского бита мы тратим больше энергии — величина E_b возрастает (без кода мы могли бы её поднять за счёт увеличения мощности).

Здесь (рис. 2) мы можем видеть падение вероятности ошибки до очень низких значений (для ОСШ выше 15 дБ), которых в реальности не будет из-за остаточного уровня ошибок (*Error Floor*) демодулятора, обусловленного фазовыми шумами приёмника, а также эффектами квантования при аналого-цифровом преобразовании (АЦП).

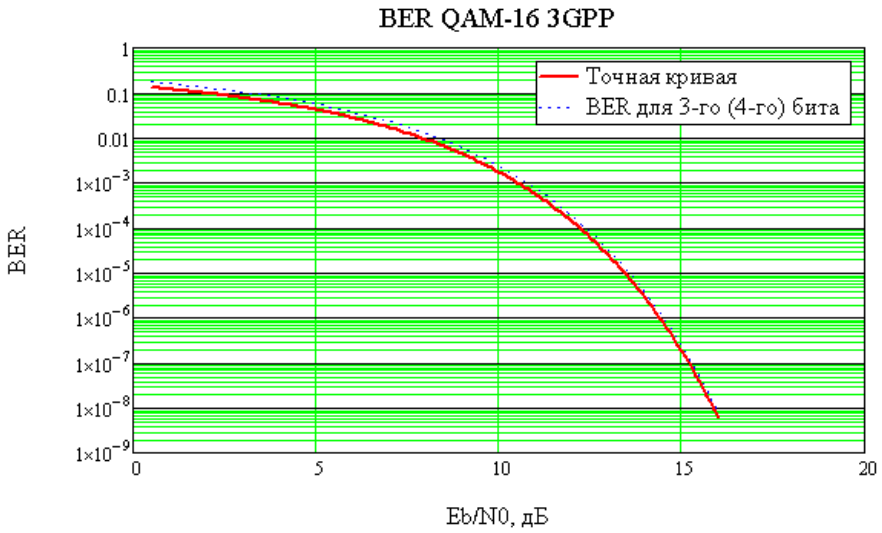


Рисунок 2 Вероятность битовой ошибки для КАМ-16 3GPP Wi-Fi.

Практика цифровых систем связи подразумевает анализ *блоковой вероятности ошибки* (*Block Error Rate*, BLER, или *Frame Error Rate*, FER), потому что в системах связи по разным причинам (протоколы разных уровней ISO/OSI, требование скорости обработки) информация передаётся блоками (фреймами), и имеет значение наличие/отсутствие ошибок во всём блоке. Блоки как правило подкрепляются отдельной контрольной суммой, которая предназначена для дополнительного контроля необходимости повторного запроса блока в случае обнаружения ошибки приёма. Битовая же вероятность ошибки, BER, — это всего лишь промежуточная характеристика, которая определяет блоковую вероятность ошибки. В предположении независимости битовых ошибок на выходе декодера:

$$BLER = 1 - (1 - BER)^n.$$

Здесь n — длина блока (в битах). Чем больше длина блока, тем хуже блоковая вероятность ошибки, поэтому роль корректирующих кодов — снизить битовую вероятность ошибки, чтобы BLER не росла так критически с ростом длины блока:

$$BLER \approx n \cdot BER, \text{ при } BER \to 0.$$

При низких BER величина BLER растёт линейно в зависимости от длины блока, однако, для хороших корректирующих кодов величина BER с ростом длины блока падает экспоненциально, что в итоге и даёт системный выигрыш кодирования.

Пример использования корректирующего кода

Рекурсивное мягкое декодирование кодов Рида-Маллера

Для иллюстрации алгебры LLR при декодировании корректирующих кодов рассмотрим коды Рида-Маллера *RM*(*r*, *m*) совместно с модуляцией КАМ-16. Проиллюстрируем рекурсивное мягкое декодирование кодов и приведём графики BLER, чтобы сравнить кодированную систему с некодированной.

Рассмотрим код Рида-Маллера *RM*(2, 6). Длина кода $n=2^6=64$, кодовое расстояние $d_к=2^{6-2}=16$, скорость кодирования $R=k/n=(1+6+15)/64=22/64\approx 0{,}344$. Модуляция КАМ-16 даёт увеличение скорости в 4 раза относительно ФМн-2, однако, корректирующий код уменьшает скорость, поэтому итоговая скорость будет составлять $4\cdot 22/64=22/16=1{,}375$ относительно некодированной ФМн-2.

Для рекурсивного декодирования кодов Рида-Маллера будем использовать представление (конструкцию) Плоткина:

$$\begin{matrix}(u,u+v)\in RM(r,m)\\ u\in RM(r,m-1)\\ v\in RM(r-1,m-1)\end{matrix}.$$

Эти формулы означают, что любой кодовый вектор кода Рида-Маллера *RM*(*r*, *m*) делится на две половинки: одна из них обозначается как *u*, а другая — как $u+v$. Так как $u+(u+v)=v$, то сумма (по модулю два) этих векторов-половинок даёт кодовый вектор *v*, принадлежащий коду Рида-Маллера $RM(r-1,m-1)$. Заметим, что этот код имеет то же самое кодовое расстояние, что и оригинальный код, потому что кодовое расстояние зависит от разности параметров (*r*, *m*) кода. Код $RM(r,m-1)$, которому принадлежит вектор *u*, будет обладать вдвое меньшим кодовым расстоянием.

При рекурсивном декодировании кода Рида-Маллера сначала идут в ветку, соответствующую коду *v* (назовём её *правой ветвью*), доходя в итоге до кода $RM(0,m-r)$ — кода с повторами (*Repetition Code*). Далее, при выходе из очередной правой ветки, берут текущий вектор $u+v$ и декодированный вектор *v*, и получают (суммой по модулю два) вторую оценку *u* дополнительно к первой (основной). Первую же оценку *u* получают, опускаясь по *левой ветке* вниз вплоть до кода $RM(r,r+1)$ — это код с одной проверкой на чётность (*Single Parity Check*, SPC). Имея две оценки вектора *u*, получают единственную оценку, и так далее поднимаются до вершины дерева, на которой формируется декодированный (очищенный от шума) вектор. Заметим, что в мягком режиме возможно исправлять ошибки в коде с одной проверкой на чётность, используя *алгоритм Вагнера*. В жёстком режиме этого сделать нельзя, поэтому в hard-decision декодере опускаются вплоть до кода «всё пространство» (*Universe Code*).

Распишем алгоритм мягкого рекурсивного декодирования кода Рида-Маллера по частям. Внутренние коды для удобства будем называть *подкодами*.

Изначально имеется вектор канальных LLR. Для рассматриваемого кода *RM*(2, 6) длина вектора должна быть равна 64, т. е. длине кода *n*. Обозначим базовую функцию декодирования как:

$$\text{rm_soft_recursive_decode}(r, m, l, o).$$

Здесь (*r*, *m*) — параметры подкода, *l* — вектор LLR для этого подкода, *o* — выходные (декодированные) биты, которые удобно хранить в формате вещественных чисел: бит 0 эквивалентен +1, бит 1 эквивалентен −1. Смысл такого выбора в том, что XOR некоторого бита с единицей инвертирует этот бит, соответственно, вещественное число −1 меняет знак некоторого числа при умножении на это число.

На первом шаге алгоритма декодирования (т. е. при первом вызове функции) заходим вправо: подкод *RM*(1, 5), однако, перед заходом требуется вычислить LLR-ы для вектора *v*. Для этого надо понять чему равен LLR битов, над которыми выполнен попарный XOR — такую операцию принято называть *Box-Plus* (*мягкое сложение*):

$$L_C = L_A \boxplus L_B.$$

Для раскрытия этой операции надо вернуться к исходным вероятностям битов, *p*(0), *p*(1), когда мы вводили понятие LLR, и затем вспомнить известную таблицу XOR: биту 0 соответствуют суммы (0+0) и (1+1), а биту 1 соответствуют суммы (0+1) и (1+0). Здесь под суммой понимается XOR (сумма по модулю 2). Обозначим первый операнд как *A*, второй операнд как *B*, а их сумму — как *C*. Исходя из теории вероятностей для независимых событий можно записать равенства:

$$p_C(0) = p_A(0)\,p_B(0) + p_A(1)\,p_B(1), \quad p_C(1) = p_A(0)\,p_B(1) + p_A(1)\,p_B(0).$$

Далее следует выразить вероятности через LLR (смотри стр. 5), что даст:

$$p_C(0) = \frac{1}{2} + \frac{1}{2}\tanh(L_A/2)\tanh(L_B/2), \quad p_C(1) = \frac{1}{2} - \frac{1}{2}\tanh(L_A/2)\tanh(L_B/2).$$

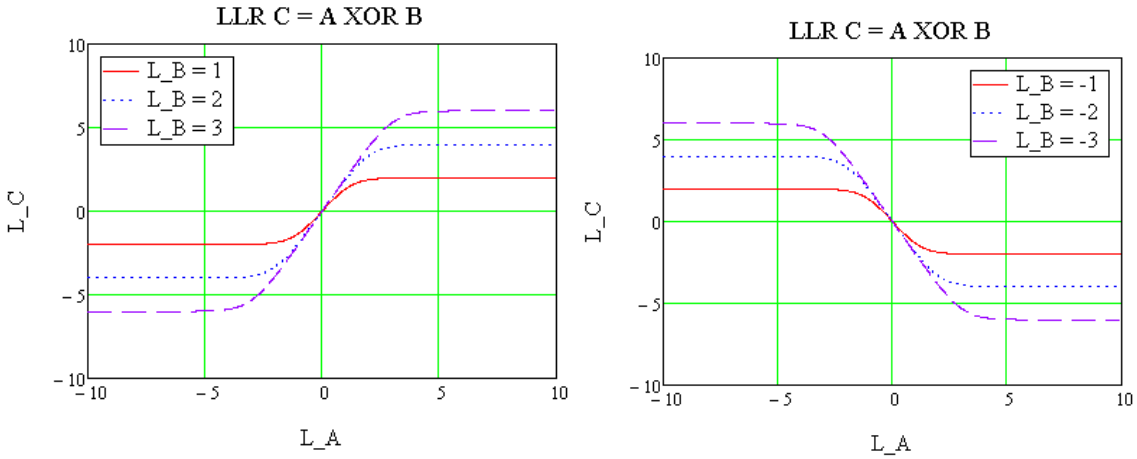
Используя известное равенство:

$$\tanh^{-1}(x) = \frac{1}{2}\ln\frac{1+x}{1-x}, \quad x \in (-1, 1),$$

получим:

$$L_C \stackrel{\text{def}}{=} \ln\left(p_C(0)/p_C(1)\right) = 2\tanh^{-1}\left(\tanh(L_A/2)\tanh(L_B/2)\right).$$

Это точное выражение для операции Box-Plus. Вид кривых LLR для этой операции показан на рисунке ниже.



(мягкое суммирование двух LLR – второй LLR показан параметром)

Можно показать, используя анализ больших и малых, что хорошо работает следующая кусочно-линейная аппроксимация мягкого сложения, называемая *min-sum*:

$$L_C \approx \operatorname{sgn}(L_A)\operatorname{sgn}(L_B)\min(|L_A|,|L_B|).$$

Для рекурсивного декодирования кодов Рида-Маллера применяют именно её в чистом виде, а для LDPC кодов из-за итеративной природы декодирования (алгоритм распространения доверия, *Belief Propagation Algorithm*) используют модификации:

1. Offset Min-Sum Algorithm (OMSA) — при низкой разрядности чисел (3-4 бита).
2. Normalized Min-Sum Algorithm (NMSA) — если позволяет разрядность (5-6 бит).

Поясним логику Box-Plus. Если L_B больше нуля, то значит, что операнд B похож на бит 0, и если при этом L_A меньше нуля (A есть бит 1), то итоговый LLR будет меньше нуля (бит 1), что согласуется с жёсткой логикой XOR: $C = A + B = 1 + 0 = 1$.

Слово *sum* в названиях алгоритмов используется по историческим причинам из-за того, что в итоговом алгоритме декодирования (что для Рида-Маллера/полярных кодов, что для LDPC) используется суммирование LLR помимо поиска минимума. Алгоритмы именуются «в целом», и было бы глупо опускать вторую не менее важную часть вычислений.

Вернёмся к описанию алгоритма рекурсивного декодирования кода Рида-Маллера *RM*(2, 6).

1. Заходим вправо: перед тем, как зайти внутрь подкода *RM*(1, 5), вычисляем LLR для вектора *v* через min-sum приближение: делим исходный вектор LLR на две части, условно *A* и *B*, и применяем min-sum формулу для каждой пары (наберётся 32 пары). Передаём вектор LLR декодеру подкода *RM*(1, 5).
2. Заходим вправо: перед тем, как зайти внутрь подкода *RM*(0, 4), кода с повторами, аналогично применяем min-sum к двум векторам из 16 отсчетов. Входим внутрь подкода *RM*(0, 4), и т. к. это код с повторами, то путём сравнения LLR с пороговым уровнем (с нулём) принимаем решение о бите, из которого состоит кодовый вектор данного кода. Принятое решение (16 одинаковых битов) пишем в выход *o*.

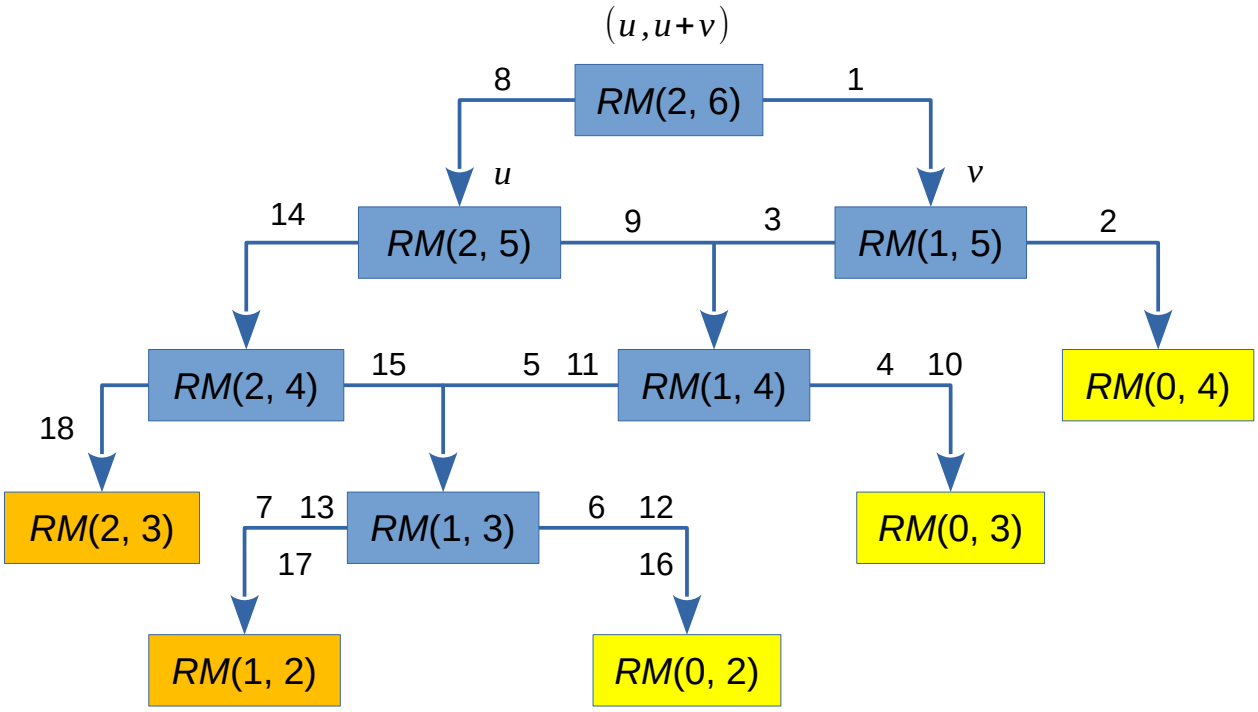


Рисунок 3: Схема рекурсивного декодирования кода Рида-Маллера RM(2, 6) в случае остановки на кодах SPC (оранжевые листья дерева) и Repetition Codes (жёлтые листья дерева). Номера 1–18 показывают последовательность обхода дерева подкодов.

3. Поднимаемся на уровень подкода *RM*(1, 5). Обновляем 16 левых отсчётов LLR, которые обозначали буквой *A*, основываясь на принятом решении *v* (на правых 16-ти битах):

   $$L_A = L_A + L_B \cdot b_B.$$

   Вспомним, что биту 0 соответствует вещественное число $b_B = +1$, а т. к. бит 0 не меняет произвольный бит операцией XOR, то значит LLR-ы должны складываться условно говоря «в фазе». Бит 1 при операции XOR инвертирует бит, значит LLR-ы должны складываться в противофазе. Здесь мы, фактически, объединяем информацию (о векторе *u*), основываясь на двух независимых источниках информации. По теореме вероятностей вероятность произведения таких (независимых) событий равна произведению вероятностей и, соответственно, сумме логарифмов вероятностей (т. е. сумме LLR).

4. И так далее. Единственный момент, который следует учесть, это то, что в рассматриваемом примере используется оптимизированный вариант рекурсивного декодирования, когда останавливаются на кодах SPC вместо кодов «всё пространство». Если этого не сделать, то диаграмма (рис. 3) содержала бы намного больше блоков. Для декодирования кода SPC использую *алгоритм декодирования Вагнера*. По имеющимся LLR путём сравнения с порогом определяют биты. Если контроль чётности не сработал, то инвертируется тот жёсткий бит, LLR которого по модулю меньше остальных (штраф при ошибочном инвертировании будет меньше). Если же контроль чётности сработал, то с жёсткими битами ничего не делается.

Таким образом, выяснили, что процедура рекурсивного декодирования кодов Рида-Маллера содержит, помимо обхода дерева в глубину (*Depth-First Search*, DFS), две основных процедуры:

- Min-Sum операцию (или любой её аналог) над LLR,
- Объединение (сложение) LLR,

а также две терминальных (т. е. на листах дерева) процедуры:

- Декодирование кодов с повторами (Repetition Codes),
- Декодирование кодов с одной проверкой на чётность (SPC).

Количество Repetition Codes равно:

$$m-2r+1, \text{ при } m \geq 2r,$$

$$r \text{ в противном случае,}$$

а количество SPC-кодов есть:

$$r, \text{ при } 2r < m,$$

$$2r-m+1, \text{ при } 2r \geq m.$$

Таким образом, общее количество листьев дерева декодирования равно $m-r+1$. Эта формула может быть использована, например, для вычисления общего количества подпрограмм декодирования (или специализированных аппаратных блоков). Высота дерева равна $m-2$. Общая сложность декодирования $O(n \log n)$, где n — длина кода. Результат численной симуляции вероятности блоковой ошибки после декодирования показан на рис. 4.

Заметим, что полярные коды декодируются похожим образом, потому что коды Рида-Маллера и полярные коды — близкие родственники: для полярных кодов также справедлива конструкция Плоткина (пусть и в транспонированном виде), однако, в полярных кодах базовый алгоритм декодирования (*Successive Cancellation*, SC) спускается вплоть до единичных битов, а также дополнительно учитывается априорная информация о «замороженности» бита[9]. За счёт оптимального выбора позиций замороженных битов (расчёт по определённому алгоритму на стадии проектирования кода) полярные коды будут выигрывать кодам Рида-Маллера в диапазоне средних ОСШ, причём чем длиннее код, тем больше выигрыш; для очень коротких кодов предпочтение следует отдать Риду-Маллеру. Для больших ОСШ коды Рида-Маллера выиграют, рис. 4, из-за главенствующей роли кодового расстояния (геометрической характеристики) в этом диапазоне. Полярная же последовательность является статистической характеристикой: она хитрым (не геометрическим) образом определяет распределение кодовых векторов с одинаковым весом Хемминга. Для кодов Рида-Маллера кодовые векторы с одинаковым весом жёстко зафиксированы (точнее, их нельзя исключить — заморозить).

Отметим, что процедуру кодирования кода Рида-Маллера можно делать рекурсивно, как и декодирование. При этом при обходе дерева и спуске до терминальных узлов (*Repetition Code* и *Universe Code*) информационные биты кодируются непосредственно, а при подъёме

9 Используется заранее вычисленная полярная последовательность (набор индексов), по которой для заданного полярного (n, k)-кода формируется маска информационных (маска 1) и замороженных (маска 0) битов.

конструируются слова в соответствии со структурой Плоткина, чтобы в итоге на вершине дерева сформировать кодовый вектор исходного кода.

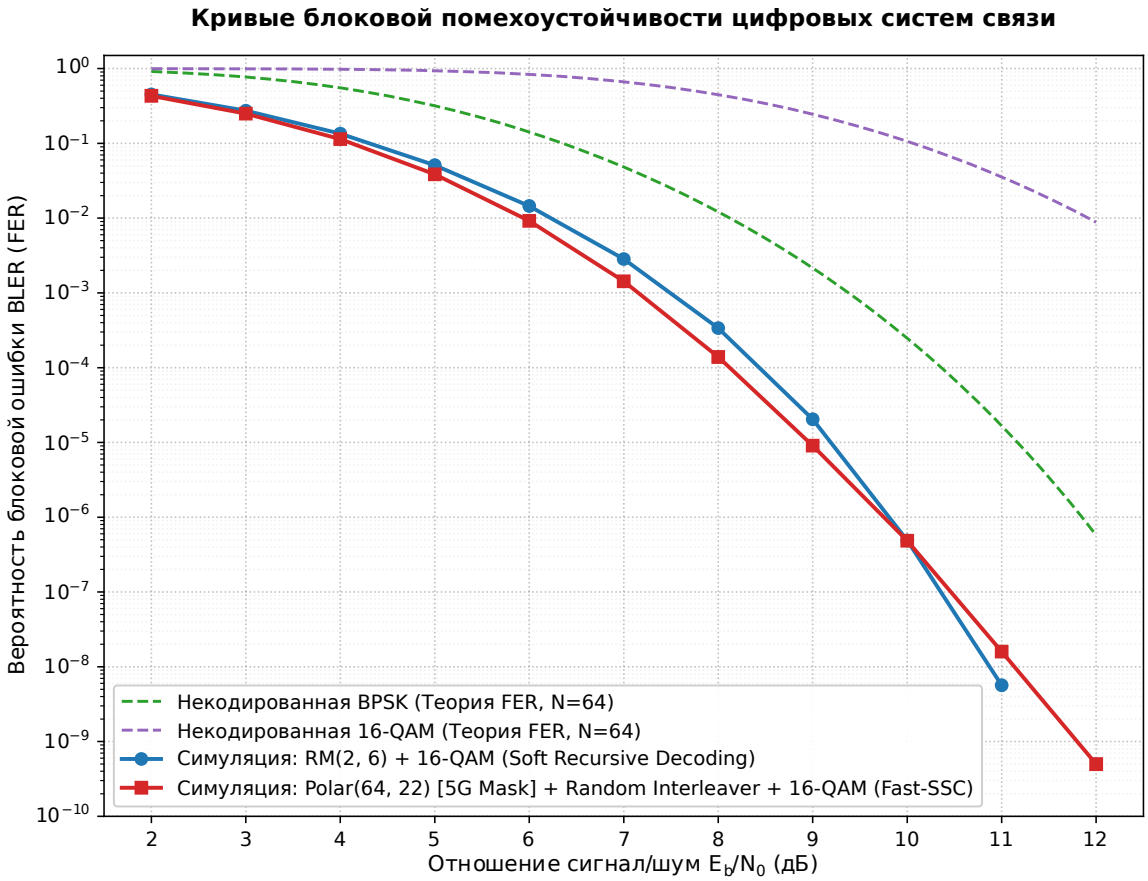


Рисунок 4 Сравнение кодированной КАМ-16 и некодированных КАМ-16 и ФМн-2 систем по блоковой вероятности ошибки при короткой длине блока (64 бита).

Видим, что для средних ОСШ полярный код (64, 22) несколько выигрывает коду Рида-Маллера (2, 6) из-за использования *полярной последовательности* (из стандарта 5G, 3GPP TS 38.212), однако, для больших ОСШ код Рида-Маллера начинает побеждать за счёт большего кодового расстояния, которое хорошо работает именно при большом ОСШ. Полярный код при высоких ОСШ страдает от так называемого Error Floor (эффекта насыщения кривой ошибки), вызванного наличием кодовых слов с малым весом Хемминга, в то время как жёсткая геометрия кодовых расстояний Рида-Маллера обеспечивает крутой наклон кривой в области высоких ОСШ. Фиксируя BLER равным $0{,}1$[10], отметим, что выигрыш полярного кода равен 0,2 дБ. Некодированная КАМ-16 при этом на 6 дБ дальше, однако у неё скорость 4 бита на символ, а для кодированной КАМ-16 скорость равна 1,375 бит/символ. Некодированная ФМн-2 на 2 дБ дальше и имеет при этом меньшую скорость — 1 бит/символ. Таким образом, **выигрыш от кодирования имеется как по скорости, так и по энергии**. Отметим, что мы рассмотрели короткие коды; **более длинные коды дадут больший выигрыш** как в плане улучшения эффекта усреднения (в полярных кодах играет роль статистика, а не геометрия кодовых расстояний), так и в плане отдаления от BLER некодированных систем, которая ухудшается с ростом длины блока (длина блока равна длине кода). С ростом длины блока возникает эффект раскрывающихся «ножниц» — зазор между кривыми BLER увеличивается.

10 Пороговое значение, выше которого начинается глобальная смена стратегии кодирования-модуляции в мобильных системах связи 4G и 5G.

Заметим, что при использовании высокоуровневой модуляции (например, КАМ-16), даже если канал без замираний, требуется битовый *интерливер* (*перемежитель*). Его задача — разрушить регулярные паттерны созвездия КАМ-М, в котором биты внутри одного символа обладают принципиально разной надежностью из-за геометрии созвездия. Универсальная полярная последовательность стандарта 5G NR рассчитывалась для AWGN-канала в предположении одинаковой надёжности входных битов, что характерно для простейших модуляций вроде ФМн-2 (BPSK) или ФМн-4 (QPSK). Если отключить интерливер, то «слабые» биты КАМ-16 попадут на позиции, которые декодер считает надёжными, что приведет к снижению помехоустойчивости. В таких условиях полярный код может уступить коду Рида-Маллера на средних ОСШ.

Декодирование полярных кодов

Алгоритм Successive Cancellation, SC

Базовые сведения

Алгоритм SC является базовым алгоритмом декодирования полярных кодов. Он идеологически совпадает с рассмотренным ранее рекурсивным декодированием кодов Рида-Маллера с точностью до транспонирования базовой матрицы, однако, «ванильный» SC-декодер погружается вглубь вплоть до одиночного бита в отличие от декодера Рида-Маллера.

Рекурсия кодов Рида-Маллера (конструкция Плоткина) задаётся базовой матрицей:

$$F_{\mathrm{RM}}=\begin{pmatrix}1 & 1\\ 0 & 1\end{pmatrix},$$

а для полярных кодов — матрицей Арикана:

$$F_{\mathrm{Polar}}=\begin{pmatrix}1 & 0\\ 1 & 1\end{pmatrix}=F_{\mathrm{RM}}{}^{T}.$$

Видно, что эти матрицы связаны операцией транспонирования[11].

Порождающая матрица, *G*, обоих кодов получается отбором необходимых строк из матрицы, порождённой *m*-й кронекеровой степенью соответствующей базовой матрицы. Для кодов Рида-Маллера отбираются строки, обладающие наибольшим хемминговым весом, причём жёстко соблюдается комбинаторный принцип: в процессе последовательного отбора, как только вес изменился, считается, что произошел переход на следующий код *RM*(*r*, *m*), т. е. поменялся параметр *r*. Для полярных кодов строки отбираются в соответствии с заранее вычисленной полярной последовательностью, которая учитывает типовой сценарий работы системы связи (например, типовое ОСШ, гауссовость шума, использование интерливера). Статистическое свойство полярных кодов таково, что разные позиции битов имеют разную степень надёжности, и для информационных битов имеет смысл выбирать самые лучшие позиции (подканалы). Благодаря этому **полярные коды являются абсолютно гибкими** в плане выбора количества информационных битов, а значит, и скорости кодирования.

В полярных кодах применяется концепция «замороженных» битов (*Frozen Bits*), на позиции которых кодер принудительно проставляет логические нули. Под «заморозку»

11 Плоткин — есть транспонированный Арикан 🐯

попадают худшие подканалы. Знание позиций этих битов помогает декодеру очистить шум: при обработке замороженной позиции декодер игнорирует канальный LLR и подставляет значение для «чистого» нулевого бита (*+inf* в соответствии со стандартной нотацией LLR), что позволяет эффективно исправлять ошибки в последующих информационных битах.

Алгоритм SC спускается вплоть до единичного бита, однако для кодов Рида-Маллера мы экономили на спусках, используя готовые формулы декодирования тривиальных кодов — кодов с повторами и кодов с одной проверкой на чётность. Аналогично, существует модификация алгоритма SC под названием Fast Simplified Successive Cancellation (Fast-SSC), в которой применяются терминальные узлы типа:

- Rate-0, все биты в поддереве заморожены,
- Rate-1, все биты информационные,
- Rep, все биты заморожены, кроме последнего информационного,
- SPC, первый бит заморожен, остальные — информационные.

Рассмотрим сначала алгоритм кодирования полярного кода.

Алгоритм кодирования полярного кода

На вход кодера поступают *k* информационных битов. Используя заранее сформированную полярную последовательность и, соответственно, маску (она показывает позиции информационных битов среди *n* кодовых битов), формируем вектор из *n* битов, в котором в соответствии с маской размещены *k* информационных битов, а остальные биты заморожены — установлены в 0.

Далее запускаем рекурсивное ядро, которое делит входной вектор пополам и делает XOR битов этих половинок, результат которого записывается в левую половину. Далее внутри запускается это же ядро для левой половины и для правой половины. Как только размер вектора будет равен одному элементу, рекурсия прекращается (это условие останова рекурсии):

```
void polar_encode_recursive_core(std::span<int> codeword) {
  const size_t n = codeword.size();
  if (n <= 1) return;
  const size_t half = n / 2;

  // x1 = u1 ^ u2, x2 = u2
  for (size_t i = 0; i < half; ++i) {
    codeword[i] = codeword[i] ^ codeword[half + i];
  }

  // Рекурсивный спуск к нижним подблокам
  polar_encode_recursive_core(codeword.subspan(0, half));
  polar_encode_recursive_core(codeword.subspan(half, half));
}
```

Рекурсивную природу кодирования можно наглядно увидеть на примере кодирования некоего вектора кодирующей матрицей, равной, например, второй кронекеровой степени матрицы Арикана:

$$\begin{pmatrix} a & b & c & d \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{pmatrix} = \begin{pmatrix} a+b+c+d & b+d & c+d & d \end{pmatrix}$$ — произведение (А).

$$\left(\begin{pmatrix} a & b \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} + \begin{pmatrix} c & d \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} \quad \begin{pmatrix} c & d \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} \right)$$ — поблочная запись произведения (А).

Здесь $\begin{pmatrix} a & b \end{pmatrix}$ и $\begin{pmatrix} c & d \end{pmatrix}$ — двухэлементные векторы-половинки, которые являются левой и правой частью рекурсии алгоритма кодирования. Если вынести матрицу Арикана 2x2 за скобки справа, то можно заметить, что левая часть итогового вектора формируется так: левые и правые части складываются и далее обрабатываются рекурсией следующего уровня — матрицей Арикана; правая же часть итогового вектора обрабатывается рекурсией 2x2.

Заметим, что если обнулить *a*, *b* и *c*, т. е. заморозить все биты, кроме последнего, то получим соответствие узлу типа Rep (в алгоритме Fast-SSC), потому кодирование такого вектора даст вектор (d, d, d, d) — именно по этой причине узел имеет тип Rep (Repetition, код с повторами). Опционально в алгоритме Fast-SSC может встретится узел типа SPC (Single Parity Check), для которого первый бит заморожен ($a=0$ в примере), а остальные биты — информационные. Такое кодирование даст кодовое слово с чётной суммой битов, потому что вычислению этой суммы соответствует сумма столбцов кодирующей матрицы при игнорировании первой строки. Количество единиц в любой строке кодирующей матрицы, кроме первой, всегда чётное — это следует из рекурсивной природы исходной матрицы Арикана.

В отличие от рекурсивного кодера Рида-Маллера *RM*(*r*, *m*), который выполняет обход дерева и сокращает (подрезает) ветви рекурсии на основе параметра *r*, полярный кодер не использует древовидную структуру. Он всегда вычисляет полный, фиксированный граф вычислений («бабочку») сложности $O(N \log N)$ независимо от количества информационных и замороженных битов.

Что касается декодирования, то в отсутствие шума повторное умножение кодированного вектора на порождающую матрицу возвращает исходный вектор битов. Это возможно благодаря уникальному свойству матрицы полярного кода в поле GF(2), где обратная матрица равна самой себе. Однако на практике, в условиях шумного канала, декодирование нельзя свести к простому матричному умножению. Вместо этого используется тот же самый топологический граф вычислений («бабочка»), но его узлы заменяются с простых логических элементов XOR на специальные функции, которые обрабатывают мягкие метрики вероятностей (LLR) для исправления ошибок.

Для того, чтобы понять алгоритм декодирования, рассмотрим простейший пример с матрицей 2x2.

Пусть некодированные биты равны (b_0, b_1), тогда в соответствии с матрицей Арикана получим канальные (кодированные) биты (b_0+b_1, b_1). После приёма будем иметь вектор из

двух LLR, (l_0, l_1), которые определяют степени уверенности принятых канальных битов. Чтобы вычислить LLR для бита b_0, следует воспользоваться мягкой суммой Box-Plus, потому что этот бит, исходя из свойств кода, равен сумме двух канальных битов.

Определения *f*- и *g*-функций

Из min-sum аппроксимации мягкой суммы следует, что степень надёжности левого бита (*левый подканал*, условно $W-$) ниже. По вычисленному LLR — эту операцию обычно называют *f*-функцией — принимается решение о левом бите, $\hat{b}_0$. Далее, поскольку информация о правом бите, b_1, есть в обоих принятых LLR, то эти LLR объединяются — эту операцию обычно называют *g*-функцией.

Эффект поляризации каналов

Допустим, принято решение $\hat{b}_0 = 0$, тогда логично, что LLR необходимо сложить друг с другом, $l_0 + l_1$, потому что в этом случае (если решение принято верно) эти LLR характеризуют два одинаковых бита (b_1, b_1). При суммировании LLR происходит **когерентное накопление сигнала**, а значит подканал для правого бита (*правый подканал*, условно $W+$) будет иметь более высокую надёжность относительно подканала левого бита — это и есть *эффект поляризации каналов*, который раскрывается полностью при неограниченном увеличении длины кода: в этом случае одна часть будет состоять из очень надёжных подканалов — их доля будет соответствовать *пропускной способности канала*, а другая часть будет состоять из очень шумных подканалов, где следует передавать замороженные биты.

Если принято решение $\hat{b}_0 = 1$, то фактически приняты биты $(1 + b_1, b_1)$, то есть левый бит инвертирован относительно правого, значит LLR-ы следует вычесть, $l_1 - l_0$. По факту здесь также сложение, но левый LLR инвертирован по знаку. Надёжность правого подканала также стала лучше.

Таким образом, за счет специального комбинирования LLR в *g*-функции полностью исключается влияние левого бита на канальное значение правого бита. Термин “cancellation” (исключение) как раз и означает очищение метрики от этого влияния с помощью уже принятого жёсткого решения. Данная процедура выполняется последовательно (successive), бит за битом, по мере продвижения по кодовому дереву.

При рекурсивном повторении описанной процедуры декодирования, для кода с большой длиной *n* произойдёт постепенное распределение подканалов по степени их надёжности, причём одни виртуальные каналы будут **статистически закономерно** становиться всё хуже и хуже с ростом длины кода (на эти позиции следует ставить замороженные биты), а другие каналы — всё лучше и лучше. Именно за математическое доказательство этого феномена Эрдал Арикан получил премию Шеннона. В этом проявляется статистическая природа полярных кодов, для которых классическое минимальное кодовое расстояние перестаёт быть определяющей метрикой. Современные длинные коды (LDPC и полярные) реализуют статистический подход взамен строго геометрического, что позволило вплотную приблизиться к пределу Шеннона, который изначально доказывался методом случайного кодирования. Главная вычислительная сложность сместилась в область дизайна таких кодов — построение эффективной

разреженной проверочной матрицы для LDPC и точный расчёт полярной последовательности для конкретного физического канала.

Алгоритм Successive Cancellation устроен так, что единичная ошибка в любом бите лавинообразно разрушает всю последующую процедуру декодирования (эффект размножения ошибок — *Error Propagation*). Именно поэтому на наименее надёжные позиции, которые из-за фрактальной структуры матрицы Арикана преобладают в первой половине блока, принудительно устанавливают замороженные биты (логические нули). В результате функция g получает на этих позициях гарантированную опору в виде априорного LLR, равного $+inf$. Когда такие замороженные биты формируют сплошные последовательные блоки (так называемые поддеревья нулевой скорости), надёжность метрик на каждом уровне рекурсии эффективно аккумулируется, благодаря чему к моменту начала информационных битов декодер успешно компенсирует влияние шума канала.

Алгоритм SC декодирования

Распишем алгоритм SC-декодирования (n, k) полярного кода на C-подобном псевдоязыке.

Обозначим ядро SC-декодера как функцию:

sc_decode_core(i, l, p, o),

где i — смещение, $0 \le i < n$, l — вектор канальных LLR, p — полярная последовательность (точнее маска из нулей и единиц), o — выходной (декодированный) вектор.

```
void sc_decode_core(i, l, p, o) {
  n = len(l)
  if (n == 1) { // Базовый случай — лист дерева.
    if (p[i] == 0) o[i] = 0
    else { if (l[0] > 0) o[i] = 0 else o[i] = 1 }
  } // if n == 1
  h = n / 2
  ll = l[0...h-1]
  lr = l[h...n-1]
  l_u1 = f(ll, lr) // Половинная длина, min-sum функция
  sc_decode_core(i, l_u1, p, o) // Будет декодирована левая половина подблока o
  l_u2 = g(ll, lr, o[i...i+h-1]) // Cancellation. Использует декодированные биты в o
  sc_decode_core(i + h, l_u2, p, o)
} // sc_decode_core(.)
```

Точкой входа в декодер будет функция, возвращающая вектор информационных битов:

```
vector sc_decode_top_level(l, p, k) {
   n = len(l)
   o = vector(n)
   sc_decode_core(0, l, p, o)
   s = vector(k) // Выходной вектор информационных бит.
   for (i = 0, j = 0; i < n; i++) {
      if (p[i] == 1) s[j++] = o[i]
   }
   return s
} //  sc_decode_top_level(.)
```

Пример полярного кода (8, 4)

Рассмотрим пример кодирования и декодирования полярного кода (8, 4) со скоростью кодирования ½. Маска, полученная по полярной последовательности для данного кода:

$$p=(0,0,0,1,0,1,1,1).$$

Нулём обозначены позиции замороженных битов, единицей обозначены позиции информационных битов. Скорость кодирования равна ½, поэтому выбраны четыре лучшие позиции для информационных битов.

Описываемые ниже шаги кодирования и затем декодирования рекомендуется параллельно рисовать в виде дерева.

Кодирование (смотри описание на стр. 17).

Формируем кодовый вектор, соблюдая маску:

$$u=(0,0,0,u_0,0,u_1,u_2,u_3).$$

Пропускаем этот вектор через полярный кодер. Заметим, что вектор u в алгоритме кодирования перезаписывается на месте (*in-place* алгоритм):

level 0 (длина 8, полудлина 8/2 = 4):

$u_{\text{left}}=(0,0,0,u_0)+(0,u_1,u_2,u_3)=(0,u_1,u_2,u_0+u_3),$

$u_{\text{right}}=(0,u_1,u_2,u_3),$

$u=(u_{\text{left}},u_{\text{right}})=(0,u_1,u_2,u_0+u_3,0,u_1,u_2,u_3).$

level 1 (длина 4, полудлина 4/2 = 2):

A:

$u_{\text{left}}=(0,u_1)+(u_2,u_0+u_3)=(u_2,u_0+u_1+u_3),\quad u_{\text{right}}=(u_2,u_0+u_3),$

$u_A = (u_{\text{left}}, u_{\text{right}}) = (u_2, u_0 + u_1 + u_3, u_2, u_0 + u_3)$,

level 2A (длина 2, полудлина 2/2 = 1):

$u_A = (u_2 + u_0 + u_1 + u_3, u_0 + u_1 + u_3, u_2 + u_0 + u_3, u_0 + u_3)$.

level 3 (длина 1): return.

B:

$u_{\text{left}} = (0, u_1) + (u_2, u_3) = (u_2, u_1 + u_3), \quad u_{\text{right}} = (u_2, u_3)$,

$u_B = (u_{\text{left}}, u_{\text{right}}) = (u_2, u_1 + u_3, u_2, u_3)$,

level 2B (длина 2, полудлина 2/2 = 1):

$u_B = (u_2 + u_1 + u_3, u_1 + u_3, u_2 + u_3, u_3)$.

level 3 (длина 1): return.

Таким образом, получили кодовый вектор:

$$u = (u_A, u_B) = (u_2 + u_0 + u_1 + u_3, u_0 + u_1 + u_3, u_2 + u_0 + u_3, u_0 + u_3, u_2 + u_1 + u_3, u_1 + u_3, u_2 + u_3, u_3).$$

Данный вектор из битов подадим на SC-декодер (в реальности это будет, естественно, вектор из канальных LLR).

SC-декодирование (смотри стр. 20).

Рассмотрим жёсткий вариант алгоритма, когда все биты известны точно. Важно помнить про маску: именно на её основе декодер будет принудительно проставлять нулевые биты в соответствующих «замороженных» позициях.

При жёстком декодировании операция *cancellation* трансформируется в простейшее сложение по модулю два, а роль правого входа зависит от состояния канала. В канале со стираниями декодер в первую очередь доверяет прямому значению, а к операции XOR прибегает только для восстановления утерянных данных. В канале же с инверсией битов (ошибками), если значения из левой и правой ветвей конфликтуют, декодер, согласно закону поляризации, отдаёт приоритет правому входу, т. к. этот подканал обладает принципиально более высокой надёжностью.

В рамках данного примера ошибки в канал вводиться не будут, поэтому будем полностью доверять правой половине.

level 0:

Делим кодовый вектор на две половинки и xor-им их:

$$u_L = (u_2 + u_0 + u_1 + u_3, u_0 + u_1 + u_3, u_2 + u_0 + u_3, u_0 + u_3) + (u_2 + u_1 + u_3, u_1 + u_3, u_2 + u_3, u_3) = \\ = (u_0, u_0, u_0, u_0)$$.

level 1:

A:

Делим вектор (u_0, u_0, u_0, u_0) на две половинки и xor-им их:

$u_L = (0, 0)$,

level 2:

Делим вектор $(0, 0)$ на две половинки и xor-им их:

$u_L = (0)$,

- на позицию 0 записываем замороженный бит 0.
- на позицию 1 записываем замороженный бит 0.

B:

Делим вектор (u_0, u_0) — правая часть вектора из 4-х элементов — на две половинки и xor-им их:

$u_L = (0)$,

- на позицию 2 записываем замороженный бит 0.
- на позицию 3 записываем правый символ u_0.

level 0:

Запускаем декодирование правой половины (offset = 4):

level 1:

A:

Делим вектор $(u_2 + u_1 + u_3, u_1 + u_3, u_2 + u_3, u_3)$ на две половинки и xor-им их:

$u_L = (u_1, u_1)$.

level 2:

Делим вектор (u_1, u_1) на две половинки и xor-им их:

$u_L = (0)$,

- на позицию 4 записываем замороженный бит 0.
- на позицию 5 записываем правую часть u_1.

B:

Делим вектор $(u_2 + u_3, u_3)$ — правую часть — на две половинки и xor-им их:

$u_L = (u_2)$,

- на позицию 6 записываем бит u_2.
- на позицию 7 записываем u_3.

Таким образом, декодировали вектор $u = (0, 0, 0, u_0, 0, u_1, u_2, u_3)$.

Ссылка на код симулятора AWGN+QAM-16+Polar/Reed-Muller

Код симулятора (C++20), в котором (не без помощи ИИ) реализованы

- Алгоритмы кодирования и мягкого декодирования полярных кодов (SC, Fast-SSC) и кодов Рида-Маллера,
- Симуляция АБГШ-канала и рассмотренной модуляции КАМ-16 вкупе с корректирующими кодами,
- Расчёт полярной последовательности методом эволюции математического ожидания, Приложение А Расчёт полярной последовательности,

доступен здесь https://github.com/nawww83/rm_soft.

20.07.26

Анатолий Новиков

Приложение А Расчёт полярной последовательности

Полярную последовательность — последовательность индексов подканалов в порядке убывания их надёжности — рассчитывают приближённым методом гауссовской аппроксимации (*Gaussian Approximation*, GA), потому что точный расчёт методом эволюции плотности вероятностей LLR (*Density Evolution*, DE) делается численно и занимает непропорционально много времени из-за сложности аппроксимации.

Расчёты делают в приближении модуляции ФМн-2 (BPSK), потому что:

- Теория полярных кодов (точнее эффект поляризации каналов) доказана для двоичного симметричного канала без памяти.
- Рассмотрение битов приводит к симметрии плотностей распределения вероятностей (ПРВ) LLR, так, что ПРВ (гауссовская) описывается всего лишь одним параметром — математическим ожиданием. Это упрощает расчёт и достаточно для практических применений.
- Функции *f*- и *g*- декодера полярного кода выведены в предположении независимости канальных LLR.
- В практических системах связи используют битовые интерливеры для нивелирования зависимости от конкретного вида модуляции.

Величина LLR для ФМн-2 рассчитывается так:

$$L(v)=\frac{2v}{\sigma^2}.$$

Здесь принято, что $v_0=1$, $v_1=-1$. Так как величина v для канала АБГШ (AWGN) распределена по гауссовскому закону со средним v_i и дисперсией σ^2, то величина LLR также имеет гауссову ПРВ, но со средним:

$$\mu_L=\frac{2v_i}{\sigma^2},$$

и дисперсией:

$$D_L=\left(\frac{2}{\sigma^2}\right)^2\overline{(v-v_i)^2}=\frac{4}{\sigma^2}.$$

При расчёте полярной последовательности **принято считать, что передаются одни нули**, поэтому среднее значение LLR равно:

$$\mu_L=\frac{2}{\sigma^2}=\frac{D_L}{2}\geq 0, \text{ или } \mu_L=4\,SNR_{\text{linear}}.$$

Здесь SNR_{linear} — отношение сигнал-шум E_s/N_0 , которое не следует путать с ОСШ E_b/N_0. В начале работы алгоритма GA величины LLR всех подканалов заполняются именно этой средней величиной — начальной LLR.

По причине связи дисперсии и среднего, ПРВ для LLR фактически характеризуется одним параметром — математическим ожиданием, поэтому данный метод можно было бы

называть *методом эволюции мат. ожидания*. Более того, при преобразованиях Арикана для хорошего канала (условно канала $W+$) свойство единственности параметра — **согласованность дисперсии и мат. ожидания** — сохраняется точно, а для плохого канала (условно канала $W-$) используют приближение, основанное на аппроксимации Чунга (Jin-Gyun Chung), которая была сделана для LDPC-кодов и улучшена для полярных кодов Трифоновым П.В. В этом приближении согласованность параметров сохраняют принудительно — это условие калибровки, которое сохраняется на всех этапах, — а форма ПРВ в плохом подканале хоть и меняется (постепенно деградирует к дельта-функции с нулевым средним), но её принудительно подставляют в интеграл в виде гауссианы. В конечном итоге точности оказывается достаточно для практических применений: относительный порядок при сортировке подканалов практически не меняется, особенно для длинных кодов. Таким образом, видим, что в методе GA имеется всего лишь одна величина, причём положительная, смысл которой заключается в некой энергии полезного сигнала в подканале — чем она больше, тем лучше надёжность подканала.

Так как в плохом подканале $W-$ величина LLR вычисляется нелинейно (f-функция):

$$L_C = 2\tanh^{-1}\left(\tanh\left(L_A/2\right)\tanh\left(L_B/2\right)\right),$$

то для вычисления математического ожидания итогового LLR — а в методе GA нас интересует только оно — используют следующий приём:

$$M\left[\tanh\left(L_C/2\right)\right] = M\left[\tanh\left(L_A/2\right)\tanh\left(L_B/2\right)\right] = \\ = M\left[\tanh\left(L_A/2\right)\right] M\left[\tanh\left(L_B/2\right)\right].$$

Для расчётов вводят вспомогательную функцию[12]:

$$\phi(x) = 1 - M\left[\tanh\left(L/2\right)\right],$$

где аргумент x — это математическое ожидание L: при раскрытии оператора мат. ожидания $M[.]$ принудительно подставляют гауссову ПРВ со средним значением x и дисперсией $2x$. Введённая функция характеризует *метрику ненадёжности* подканала (убывающая функция): если $L=0$, то его ненадёжность равна 1, а надёжность равна 0 — худший канал; если L стремится к бесконечности, то ненадёжность равна 0, а надёжность равна 1 — лучший канал.

Тогда итоговую ненадёжность плохого подканала можно записать так:

$$\phi(\mu_C) = 1 - (1 - \phi(\mu_A))(1 - \phi(\mu_B)).$$

Для возврата из области «ненадёжности» $(0, 1]$ в область LLR $[0, \infty)$ используют обратную функцию:

$$\mu_C = \phi^{-1}\left[1 - (1 - \phi(\mu_A))(1 - \phi(\mu_B))\right].$$

Прямую функцию вычисляют аппроксимацией Трифонова-Чунга (есть разные вариации), а обратную функцию можно вычислять бисекцией, используя прямую функцию, потому что последняя монотонно убывает.

Как только определены LLR для каждого бита — листа дерева, переходят к вычислению вероятности ошибки, считая, что LLR имеет гауссову ПРВ:

12 Есть некоторые разногласия по поводу этой функции: иногда используют функцию, дополненную до единицы.

$$p_e = Q\left(\sqrt{\frac{\mu}{2}}\right),$$

где

$$Q(x) = \frac{1}{\sqrt{2\pi}} \int_x^\infty \mathrm{e}^{-t^2/2} dt.$$

Разберём нюанс, связанный с реализацией расщепления каналов в методе GA.

Допустим, длина полярного кода равна 4. Кодирование битов $\vec{b}$ матрицей Арикана 4x4 даёт следующий вектор:

$$(b_0+b_1+b_2+b_3, b_1+b_3, b_2+b_3, b_3).$$

Чтобы декодировать такой вектор (в котором нет ошибок), необходимо рекурсивно делить вектор на половины, XOR которых записывать в левую часть, а правую — просто копировать:

$b_0+b_1+b_2+b_3$	b_1+b_3	b_2+b_3	b_3
b_0+b_1	b_1	b_2+b_3	b_3
b_0	b_1	b_2	b_3
2 xor	1 xor	1 xor	0 xor

Таким способом биты будут декодированы в натуральном порядке (0, 1, 2, 3). Чем меньше операций XOR[13] было использовано, тем надёжнее подканал (бит).

Сначала декодируется бит b_0, а затем, используя LLR-ы уровнем выше, которые символически обозначены как b_0+b_1 и b_1, декодируется бит b_1. Так как в методе GA передаются одни нули, то LLR-ы при декодировании правых частей всегда следует складывать. Эти LLR получены операцией XOR от **одинаковых канальных LLR**, поэтому они будут равны и при сложении фактически возникнет удвоение. К моменту декодирования битов $b_0 = b_1 = 0$, обозначения LLR на самом верхнем уровне представленной таблицы можно упростить до: b_2+b_3, b_3, b_2+b_3, b_3. Видим, что логически имеется избыточность, подобно избыточности при декодировании бита b_1: опять возникает удвоение LLR, поэтому для правого дерева в алгоритме GA верно:

$$\mu(W+) = 2\mu,$$

а для левого — происходит операция типа:

$$\mu(W-) = \phi^{-1}\left[1-(1-\phi(\mu))^2\right].$$

Еще один нюанс полярных кодов — недостаточность такой метрики как количество xor-ов для определения надёжности подканала. На примере таблицы выше видим, что есть подканалы с равным количеством xor-ов (по единичке): вот этим-то и объясняется необходимость более тонкого расчёта для определения надёжности.

Код (C++20) для расчёта полярной последовательности приведён в репозитории https://github.com/nawww83/rm_soft.

13 Помним, что для LLR используется *f*-функция, а не XOR, но для удобства иногда можно называть это xor-ом.

Ниже показан результат расчёта полярной последовательности для полярного кода (16, 8), а также log-профиль надёжности подканалов.

```
======================================================================
   ПОЛЯРНАЯ ПОСЛЕДОВАТЕЛЬНОСТЬ (МЕТОД GA, аппроксимация Трифонова П.В.)
======================================================================
Параметры: N = 16, K = 8, Скорость R = 0.5000, Design SNR = 2.00 dB
Маска подканалов в естественном порядке (0..N-1):
0000000101111111
Rank    Index     Binary      M[LLR]          Prob. error         Type
----------------------------------------------------------------------------
1       0         0000        0.43639         0.32021             FROZEN
2       1         0001        2.42563         0.13539             FROZEN
3       2         0010        3.18824         0.10337             FROZEN
4       4         0100        4.27764         0.07181             FROZEN
5       8         1000        5.78876         0.04444             FROZEN
6       3         0011        10.14595        0.01215             FROZEN
7       5         0101        12.65316        0.00595             FROZEN
8       6         0110        14.64829        0.00340             FROZEN
9       9         1001        15.99278        0.00234             INFO
10      10        1010        17.94895        0.00137             INFO
11      12        1100        20.20925        0.00074             INFO
12      7         0111        34.31335        0.00002             INFO
13      11        1011        40.98935        0.00000             INFO
14      13        1101        45.54922        0.00000             INFO
15      14        1110        50.71658        0.00000             INFO
16      15        1111        101.43316       0.00000             INFO
----------------------------------------------------------------------------

======================================================================
          LOG-ПРОФИЛЬ НАДЕЖНОСТИ ПОДКАНАЛОВ ПО ВЕРОЯТНОСТИ ОШИБКИ
======================================================================
Метрика шкалы: -log10(error_prob). Чем длиннее шкала, тем меньше ошибок.
Условные обозначения: [.] - FROZEN канал, [#] - INFO канал
----------------------------------------------------------------------------
Ch 0   (0000) Pe: [3.20e-01] | .
Ch 1   (0001) Pe: [1.35e-01] | ..
Ch 2   (0010) Pe: [1.03e-01] | ...
Ch 3   (0011) Pe: [1.22e-02] | ......
Ch 4   (0100) Pe: [7.18e-02] | ...
Ch 5   (0101) Pe: [5.95e-03] | .......
Ch 6   (0110) Pe: [3.40e-03] | ........
Ch 7   (0111) Pe: [1.72e-05] | ###############
Ch 8   (1000) Pe: [4.44e-02] | ....
Ch 9   (1001) Pe: [2.34e-03] | ########
Ch 10  (1010) Pe: [1.37e-03] | #########
Ch 11  (1011) Pe: [2.99e-06] | ##################
Ch 12  (1100) Pe: [7.39e-04] | ##########
Ch 13  (1101) Pe: [9.11e-07] | ###################
Ch 14  (1110) Pe: [2.38e-07] | ####################
Ch 15  (1111) Pe: [5.34e-13] | ########################################
----------------------------------------------------------------------------
```

Log-профиль надёжности наглядно демонстрирует, что поляризация носит экспоненциальный характер: надёжные каналы превосходят зашумлённые не на проценты, а на много порядков, обеспечивая тем самым колоссальный энергетический выигрыш всей кодовой системы при росте длины блока. Итоговый показатель BER в системах с полярными кодами и SC-декодированием определяется теми редкими, катастрофическими событиями,

когда случайный шум в канале связи оказывается настолько аномально сильным, что «пробивает» энергетическую защиту даже хорошего подканала. При SC-декодировании полярного кода ошибки распределены в пространстве неравномерно: система либо декодирует весь блок идеально (BER = 0), либо, в редких случаях сильного шума, сваливается в лавину, выдавая пачку (*burst*) ошибок в пределах одного блока. Именно частота этих редких «сваливаний» и определяет итоговый средний BER системы.

Приложение Б Математика циклического префикса. MMSE эквалайзер и согласованный фильтр

Покажем, что добавление циклического префикса достаточной длины к формируемому сигналу передатчика превращает линейную свёртку, которой естественным образом подвергается сигнал, проходя через линию связи[14], в круговую (циклическую), что позволяет использовать простую математику в частотной области (поточечное умножение) для коррекции канальных искажений (*Frequency-Domain Equalization*, FDE).

Циклический префикс — это часть сигнала, взятая с конца и путём параллельного переноса вставленная в его начало. Применяется в OFDM системах как их естественная и неотъемлемая часть, а также в Single Carrier системах с частотно-областным выравниванием (SC-FDE).

Для полной эквивалентности линейной свёртки круговой необходимо выполнить два условия:

1. Длительность (длина) циклического префикса, $T_{\text{ср}}$, должна быть больше длительности *импульсной характеристики* (ИХ) канала распространения сигнала.
2. Начальная часть принятого сигнала длительностью $T_{\text{ср}}$ должна быть проигнорирована приёмником (отброшена).

Запишем линейную свёртку[15]:

$$v(t)=\int_{-\infty}^{t} s_{\text{ср}}(\tau)h(t-\tau)\,d\tau.$$

Здесь ИХ канала обозначена как $h(t)$; она «развивается» (т. е. не равна нулю) от нулевого момента времени, поэтому подынтегральное выражение может быть ненулевым только при $t-\tau\geq 0$ и, значит, верхний предел интегрирования достаточно взять равным t.

Положим, что сигнал $s_{\text{ср}}(t)$ уже содержит циклический префикс, имеет длительность $T+T_{\text{ср}}$, и «развивается» от момента времени $t=-T_{\text{ср}}$, поэтому:

$$v(t)=\int_{t-T_{\text{ср}}}^{t} s_{\text{ср}}(\tau)h(t-\tau)\,d\tau,\quad -T_{\text{ср}}<t<T+T_{\text{ср}}.$$

Здесь указан полный интервал, на котором будет наблюдаться сигнал в соответствии с теорией линейной фильтрации.

Отбрасывание начальной части принимаемого сигнала $v(t)$ (смотри условия эквивалентности выше) равнозначно формированию следующего сигнала длительностью T:

$$v'(t)=\int_{t-T_{\text{ср}}}^{t} s(\tau+T)h(t-\tau)\,d\tau,\ 0<t<T.$$

Заметим, что интервал T — период круговой свёртки, а сигнал $s(t)$ — сигнал без циклического префикса, «развивающийся» от нулевого момента времени. Наблюдаемый сигнал $v'(t)$ взят строго на интервале $(0,T)$: ровно так, как это будет сделано при обработке

14 Актуально для радиолиний связи; в меньшей степени для кабельных линий. Линию связи вне зависимости от её природы можно называть каналом распространения сигнала.

15 По правилу «трёх тау», которое запоминается буквально скороговоркой: тау—тэ-минус-тау—дэ-тау.

в блоке FDE. Здесь для справедливости интеграла предполагается, что сигнал $s(t)$ периодический с периодом *T*.

При цифровой обработке в блоке FDE используется *дискретное преобразование Фурье*, ДПФ, которое **неявным** образом периодически продолжает сигнал, что входит в противоречие с физикой реального канала, который сигналы никаким образом не продолжает, но оказывается, что продолжить их можно искусственно циклическим префиксом. Чем «проще» канал, тем короче его ИХ, и тем короче допускается брать префикс.

Напомним, что круговая свёртка ИХ некоего канала с неким сигналом $s(t)$ выглядит следующим образом:

$$w(t)=\int_0^T s(\tau)h(t-\tau)d\tau.$$

Данная свёртка по определению периодическая функция по *t* с периодом *T*, причём как сигнал, так и ИХ — периодические функции с периодом *T*.

Заметим, что периодическое продолжение ИХ канала — это математическое следствие того, что передаваемый сигнал содержит циклический префикс.

Перепишем $v'(t)$:

$$v'(t)=\int_{t-T_{\text{ср}}}^{t} s(\tau)h(t-\tau)d\tau,\ 0<t<T.$$

Разберём до какой степени допускается расширить пределы интегрирования.

1. Так как ИХ канала дополняется нулями и имеет период *T*, то допускается брать отрицательный аргумент $t-\tau$ не меньше $-T+T_{\text{ср}}$, а т. к. рассматривается сигнал только для $0<t<T$, то допускается расширить верхний предел интегрирования для τ до $T-T_{\text{ср}}$.
2. Так как ИХ имеет длительность не выше $T_{\text{ср}}$, то допускается взять нижний предел интегрирования $-T_{\text{ср}}$.

Тогда интеграл свёртки можно записать так:

$$v'(t)=\int_{-T_{\text{ср}}}^{T-T_{\text{ср}}} s(\tau)h(t-\tau)d\tau,\ 0<t<T.$$

Заметим, что разница между пределами интегрирования равна периоду *T*, а так как сигнал и ИХ — периодические, то таким образом получили численную эквивалентность принятого с вырезанным циклическим префиксом сигнала результату прохождения сигнала без префикса через некий «циклический» канал распространения, коэффициенты передачи которого, найденные совпадают с коэффициентами передачи реального канала.

Для закрепления понимания использованного математического трюка рассмотрим пример дискретных сигналов небольшой длительности.

Пусть исходный сигнал представлен вектором

$$\vec{x}=(x_0,x_1,x_2,x_3),$$

а ИХ канала имеет вид:

$$\vec{h} = (h_0, h_1).$$

Тогда операцию прохождения сигнала через такой канал — линейную свёртку — можно записать в матричном виде:

$$\vec{y} = \begin{pmatrix} h_0 & 0 & 0 & 0 \\ h_1 & h_0 & 0 & 0 \\ 0 & h_1 & h_0 & 0 \\ 0 & 0 & h_1 & h_0 \\ 0 & 0 & 0 & h_1 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix}.$$

Видим, что матрица не является циркулянтной, потому что не все её строки являются циклическим сдвигом друг друга; матрица является тёплицевой — каждая диагональ содержит одинаковые элементы.

Известно, что для компенсации канальных искажений достаточно взять длину циклического префикса равной длине ИХ канала без единицы, поэтому для рассматриваемого примера достаточно взять префикс из одного отсчёта. Пропустим такой сигнал через физический канал:

$$\vec{v} = \begin{pmatrix} h_0 & 0 & 0 & 0 & 0 \\ h_1 & h_0 & 0 & 0 & 0 \\ 0 & h_1 & h_0 & 0 & 0 \\ 0 & 0 & h_1 & h_0 & 0 \\ 0 & 0 & 0 & h_1 & h_0 \\ 0 & 0 & 0 & 0 & h_1 \end{pmatrix} \begin{pmatrix} x_3 \\ x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix}.$$

Теперь, если «выкусить» первый отсчёт принятого сигнала и взять ровно столько отсчётов, сколько содержит исходный сигнал (без префикса, т. е. в данном примере — 4 отсчёта), то получившиеся уравнения можно переписать в матричном виде так:

$$\vec{v}' = \begin{pmatrix} h_0 & 0 & 0 & h_1 \\ h_1 & h_0 & 0 & 0 \\ 0 & h_1 & h_0 & 0 \\ 0 & 0 & h_1 & h_0 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix}.$$

Полученная матрица является циркулянтной (циркулянтом), потому что каждая последующая строка матрицы есть циклический сдвиг текущей строки. Таким образом, добавление к исходному сигналу циклического префикса достаточной длины даёт эффект прохождения исходного сигнала через «циклический» канал, что позволяет применить операцию ДПФ[16] к принятому вектору $\vec{v}'$:

$$\begin{bmatrix} V_0' \\ V_1' \\ V_2' \\ V_3' \end{bmatrix} = \begin{bmatrix} H(0) & 0 & 0 & 0 \\ 0 & H(1) & 0 & 0 \\ 0 & 0 & H(2) & 0 \\ 0 & 0 & 0 & H(3) \end{bmatrix} \begin{bmatrix} X_0 \\ X_1 \\ X_2 \\ X_3 \end{bmatrix},$$

16 Известно, что ДПФ приводит циркулянт к диагональному виду. Также можно воспользоваться теоремой о свёртке.

и таким способом найти спектральные коэффициенты $H(k)$ канала передачи, а значит — скорректировать искажения.

Чтобы получить формулу корректора, учтём наличие шума $N(k)$ в канале, тогда в спектральной области в приёмнике можно наблюдать отсчёты (**комплексные**) вида:

$$V(k)=H(k)X(k)+N(k).$$

Стоит задача найти корректирующие коэффициенты $W(k)$ так, чтобы минимизировать ошибку:

$$E(k)=W(k)V(k)-X(k).$$

Так как для каждого *k* вычисления идут независимо (не влияют друг на друга), то достаточно оптимизировать один случай, убрав *k* из анализа. Для удобства обозначим переменные строчными (малыми) буквами. В процессе анализа учтём, что шум случайный с нулевым средним и известной дисперсией (мощностью), а сигнал с точки зрения статистической радиотехники имеет нулевое среднее значение (по ансамблю) и постоянную среднюю мощность (дисперсию). Минимизации подлежит средний квадрат ошибки (*Minimum Mean Square Equalization*, MMSE):

$$\overline{|e|^2}=\overline{[w(h\cdot x+n)-x][w^*(h^*\cdot x^*+n^*)-x^*]}.$$

Раскроем скобки и применим оператор математического ожидания к слагаемым, учтя нулевые средние сигнала и шума:

$$\overline{|e|^2}=w\,w^*|h|^2P_x-w\,h\,P_x+w\,w^*P_n-w^*h^*P_x+P_x.$$

Здесь $P_x=\overline{|x|^2}$ — средняя мощность сигнала, $P_n=\overline{|n|^2}$ — средняя мощность шума. Введём отношение сигнал-шум:

$$SNR=\frac{P_x}{P_n}.$$

Для минимизации определённого выше среднего квадрата ошибки, зависящего от комплексного коэффициента *w*, целесообразно использовать аппарат *производных Виртингера*. Данный подход позволяет найти оптимум в компактном виде, избегая громоздкого раскрытия вещественной и мнимой частей коэффициента *w*:

$$\frac{\partial\overline{|e|^2}}{\partial w^*}=w|h|^2P_x+w\,P_n-h^*P_x.$$

Заметим, что дифференцирование выполняется именно по комплексно-сопряжённому коэффициенту w^*, в то время как сам коэффициент *w* на время вычисления производной полагается константой. Из теории комплексной оптимизации известно, что такая производная по структуре и результату эквивалентна классическому вещественному градиенту — вектору частных производных, полученному при представлении комплексного коэффициента в виде двумерного вектора $(\Re(w),\Im(w))^T$.

Приравнивая полученный градиент к нулю, находим оптимальное значение коэффициента MMSE-эквалайзера:

$$w = \frac{h^*}{|h|^2 + \frac{1}{SNR}}.$$

Или, переходя к исходным переменным:

$$W(k) = \frac{H^*(k)}{|H(k)|^2 + \frac{1}{SNR}}.$$

Видим, что найденный MMSE эквалайзер автоматически содержит множитель (в числителе дроби), отвечающий за классическую согласованную фильтрацию: он будет тем более выраженным, чем выше уровень шума по отношению к сигналу.

При хорошем ОСШ коэффициенты эквалайзера могут быть аппроксимированы так:

$$W(k) \approx \frac{1}{H(k)}, \quad SNR \to \infty,$$

что выделяет множитель, отвечающий за коррекцию *частотной характеристики* (ЧХ) канала, причём сквозной характеристики, т. е. учитывая также и формирующий фильтр передатчика (например, *Root-Raised Cosine filter,* RRC):

$$H(k) = RRC(f) H_{\text{channel}}(f).$$

При плохом ОСШ коэффициент эквалайзера может быть представлен в виде:

$$W(k) = H^*(k) \cdot SNR, \quad SNR \to 0,$$

что выделяет множитель, отвечающий за энергетический сбор сигнала: происходит коррекция (пусть и слабых) гармоник по фазе, а значит, и по задержке, что как бы собирает сигнал в фокус (уровень шума при этом остаётся неизменным). Это согласованная фильтрация, но только согласование здесь — с каналом. По амплитуде гармоники не выравниваются, то есть замирания не корректируются[17], потому что шум большой и «замершие» гармоники утонули в нём: гораздо выгоднее выровнять только лишь фазы, тем более, что шум инвариантен к фазовой коррекции. В этом случае (плохое ОСШ) останется МСИ, которая вызвана неполной компенсацией амплитудных искажений частотно-селективного канала. Задержки многолучёвости будут выровнены — настолько, насколько точно оценён канал.

17 Даже усугубляются, следуя принципу подавления недостоверных гармоник, приводя LLR ближе к нулю, что информационно поможет декодеру помехоустойчивого кода.